

CSAI 203 - Fall 2025

EduTrack

Design Document

Team Number:29

Team Members:3

- *Habiba Mohammed-202402213*
- *Ahmed Mahmoud-202401491*
- *Mohamed Essam-202401171*

Representative Contact: *s-ahmed.abda@zewailcity.edu.eg*

Date: *22/11/2025*

Table of Contents:

1. Introduction

- 1.1 Purpose of the Document
- 1.2 Scope of the Design Phase
- 1.3 Intended Audience
- 1.4 Overview of the Contents

2. System Overview

- 2.1 Brief Description of the System
- 2.2 Key Design Goals and Constraints

3. Architectural Design

- 3.1 System Architecture Diagram
- 3.2 Discussion of Architectural Style and Components
- 3.3 Technology Stack and Tools

4. Detailed Design

- 4.1 Model–View–Controller (MVC) Design Pattern
 - 4.1.1 Description of MVC Pattern
 - 4.1.2 Mapping of Project Components to MVC
 - 4.1.3 Responsibilities of Model, View, and Controller
 - 4.1.4 Interaction Between Components
- 4.2 UML Diagrams
 - 4.2.2 Detailed Class Diagram
 - 4.2.3 Sequence Diagrams
- 4.3 UI/UX Design
 - 4.3.1 Wireframes / Mockups
- 4.4 Data Design
 - 4.4.1 Database Schema / ER Diagram
 - 4.4.2 File Structure / Data Storage Model (if applicable)
 - 4.4.3 Data Dictionary

5. Conclusion

- 5.1 Summary of Design Phase

1. Introduction

1.1 Purpose of the Document

This design document serves as the definitive technical plan for the development of EduTrack Smart Classroom system. It translates the approved requirements outlined in the Software Requirements Specification (SRS) into practical design artifacts, including architectural diagrams, UML class diagrams, sequence diagrams, data schema, and interface mock-ups. The document will assist developers, testers, and stakeholders towards the implementation.

1.2 Scope of the Design Phase

The design phase includes the overall system architecture, the internal design of modules, mapping the pieces to the MVC, sequence diagrams for the significant use case, detailed class diagram, the database schema, and UI wireframes. This document is focused on design

1.3 Intended Audience

- Development team
- Test team
- Stakeholders involved in maintaining or extending the system

1.4 Overview of the Contents

This document contains:

- System overview
- Architectural design and technology stack
- MVC component mapping
- UML diagrams (class + sequence)
- UI wireframes
- Database schema and data dictionary
- Final design summary

2. System Overview

2.1 Brief Description of the System

EduTrack is an online smart classroom system created to improve communication and simplify academic processes between students and faculty. For example, it supports creating courses, uploading lectures, creating and submitting assignments, grading assignments, tracking attendance, chat messaging, a dashboards and notifications.

2.2 Key Design Goals and Constraints

Design Goals:

- MVC separation of concerns
- Scalability and modularity
- Secure handling of authentication
- Reliable file upload and storage
- Reusability and simple UI

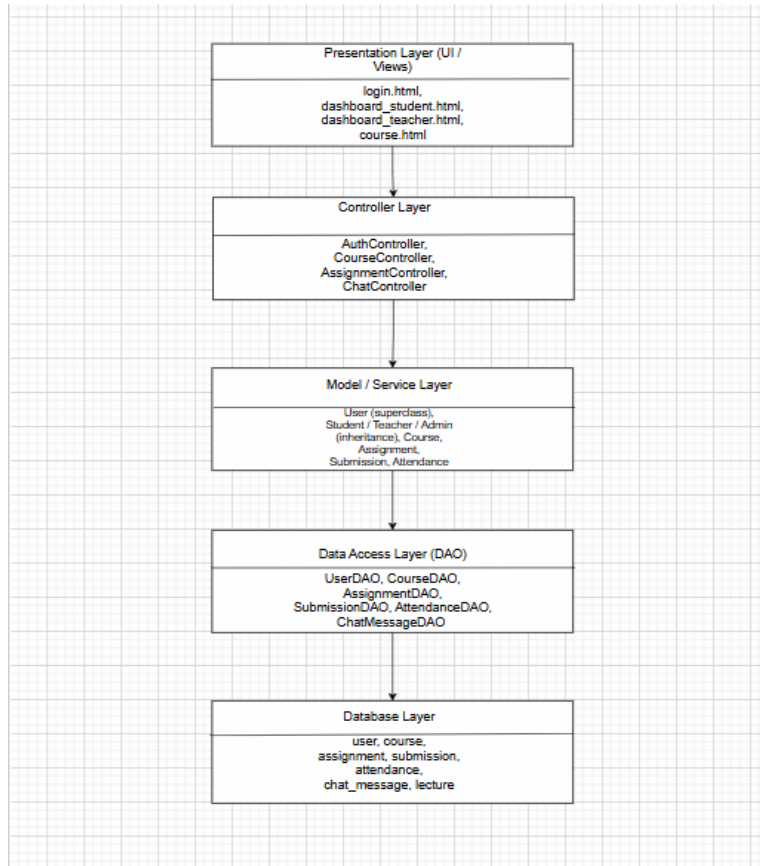
Constraints:

- Flask + SQL backend
- HTML+CSS frontend
- Max file upload: 10MB

3. Architectural Design

3.1 System Architecture Diagram

<https://drive.google.com/file/d/1ysKettSsG1R-1rTDJlhqi-iBG-pU8Fr8/view?usp=sharing>



3.2 Discussion of Architectural Style and Components

EduTrack utilizes a Monolithic Model–View–Controller (MVC) architectural style. In a monolithic system, all aspects of the application, including views, controllers, models, and data access logic, are combined and deployed together as a single entity. This enables a very simplistic structure that is easy to develop and suitable for the scope of this project.

MVC architecture:

Views (Presentation Layer) These are the HTML/Jinja templates (such as login page, dashboards, course pages) that will present information to users and capture input.

Controllers (Controller Layer)

This is accomplished with Flask route functions. Controllers will receive user requests, authenticate requests, validate input, call the model/service functions, and return the appropriate view. Examples of controllers include AuthController, CourseController, AssignmentController, and ChatController.

Models / Services (Model Layer)

These represent the lifecycle of the system's main entities and business logic, such as User, Student, Teacher, Course, Assignment, Submission, Attendance, and ChatMessage. They have a duty of enforcing business rules, such as due dates, grading, or attendance status.

DAOs (Data Access Layer)

Data Access Objects are responsible for all interaction with the SQL database. DAOs complete CRUD functions and keep SQL queries separate from the business logic. Examples include UserDAO, CourseDAO, AssignmentDAO, and AttendanceDAO.

Overall, this architectural style clearly separates the responsibilities, producing a system that is more maintainable, organized, and easy to develop as a monolithic deployment.

3.3 Technology Stack and Tools

- Backend: Python (Flask)
- Frontend: HTML, CSS
- Database: SQL (SQLite / MySQL)
- Tools: Draw.io for Detailed class diagram , sequence diagram , mock-ups ,system architecture diagram

4. Detailed Design

4.1 Model–View–Controller (MVC) Design Pattern

4.1.1 Description of MVC Pattern

MVC divides the system into:

- Model — Manages data and business logic
- View — Responsible for UI rendering
- Controller — Handles requests, processes user actions, updates the model, and selects the view

EduTrack was built using the MVC pattern because of its clear separation of concerns, making the system easy to understand, develop, and maintain. The MVC pattern separates components to reduce coupling. The MVC pattern also

allows development to happen parallelly because it separates the user interface (UI) from the backend logic. The MVC pattern is natural fit for Flask web applications, which are inherently controller-based with routing. MVC keeps the EduTrack system modular and scalable, which will make it easier to extend with features in the future.

4.1.2 Mapping of Project Components to MVC

Model Classes

- User → Student, Teacher, Admin (inherit from User in design)
- Course
- Enrollment
- Lecture
- Assignment
- Submission
- Attendance
- ChatMessage

Controller Classes

- **AuthController** – login, logout, register
- **CourseController** – create/join course, view course
- **LectureController** – upload/view lectures
- **AssignmentController** – create assignments, submit assignments
- **AttendanceController** – record & view attendance
- **ChatController** – send and display chat messages
- **AdminController** – manage users & system settings

Views

- login.html
- register.html
- dashboard_teacher.html
- dashboard_student.html
- course.html
- lectures.html
- assignments.html
- attendance.html
- chat.html
- admin_panel.html

4.1.3 Responsibilities of MVC Components

Model Responsibilities

The design incorporates Student, Teacher, and Admin as subclasses that inherit from base class User. All of them lie within the Model layer, representing different classes of system users that share the same fundamental attributes.

- Store system data
- Apply rules such as due date validation, grade constraints
- Map to database tables

View Responsibilities

- Display pages to users
- Provide forms for input (examples:upload assignment, create course)

Controller Responsibilities

- Authenticate users
- Handle course creation, joining
- Manage lecture upload
- Manage assignments and submissions
- Process chat messages
- Record attendance

4.1.4 Interaction Between Components

Overall Flow of the MVC Design Pattern

User interacts with a View

(as an example, submitting a form, clicking a button, or navigating to a page).

Controller receives the request

Controller will validate any input, check for authorization, and decide what action needs to be completed.

Controller communicates with Model

Model applies business rules (for example: validate due dates, calculate attendance status, verify user role, etc.).

Model accesses data using the DAO layer

DAOs perform SQL actions such as SELECT, INSERT, UPDATE, or DELETE.

The database returns data to the DAO → Model → Controller

Controller selects View

(HTML/Jinja template) and receives any data to display.

View renders the response to the user closing the loop.

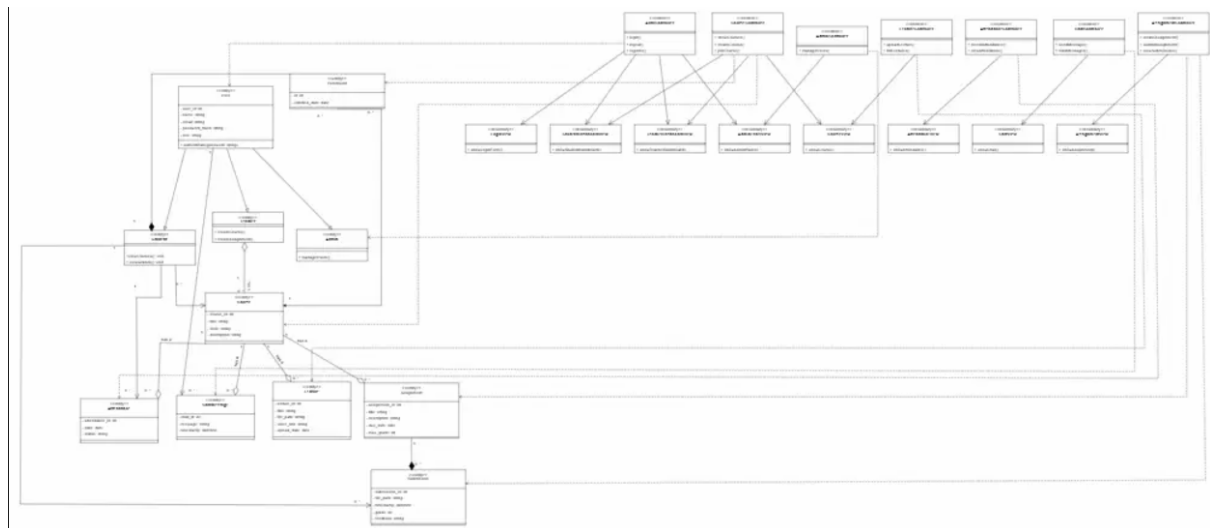
Example: "Submit Assignment"

1. View displays submission form
2. Controller receives file
3. Model validates file
4. DAO saves record in DB
5. View confirms success

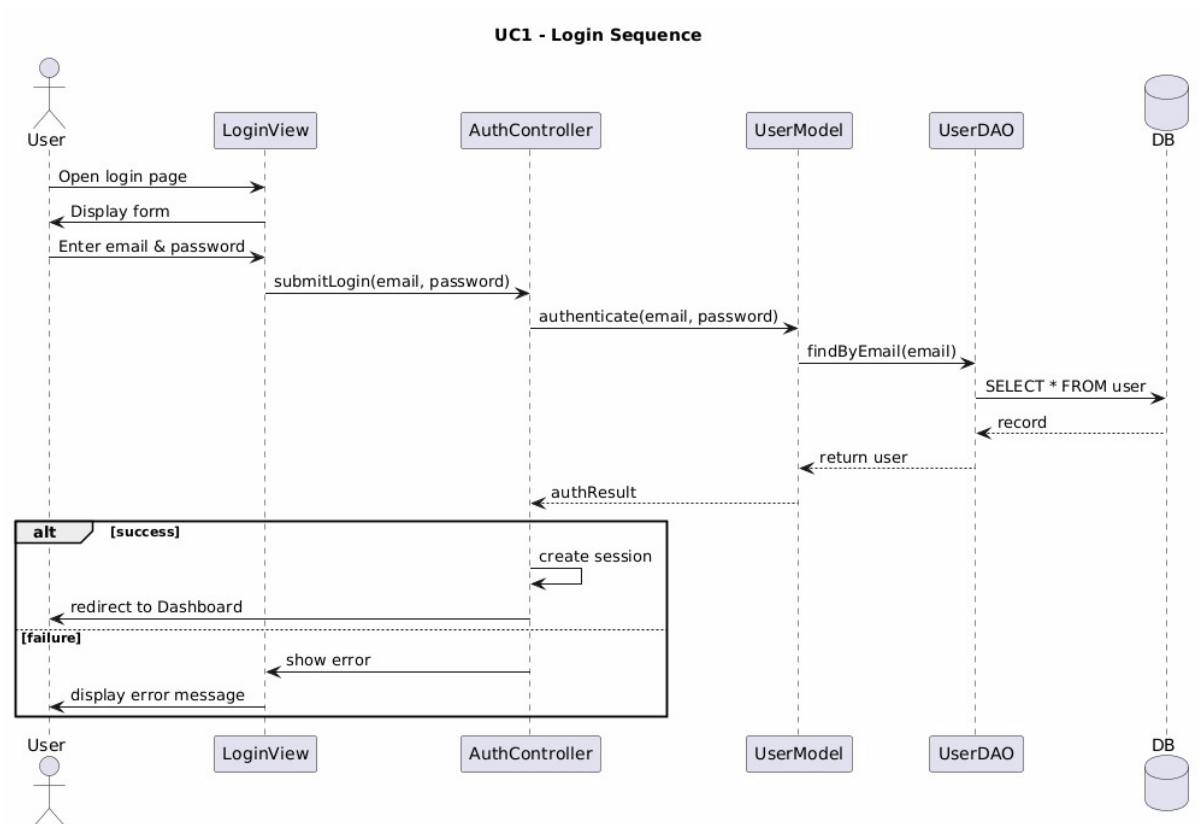
4.2 UML Diagrams 4.2.2 , 4.2.3 , 4.3.1 , 4.4.1

4.2.2 Detailed Class Diagram

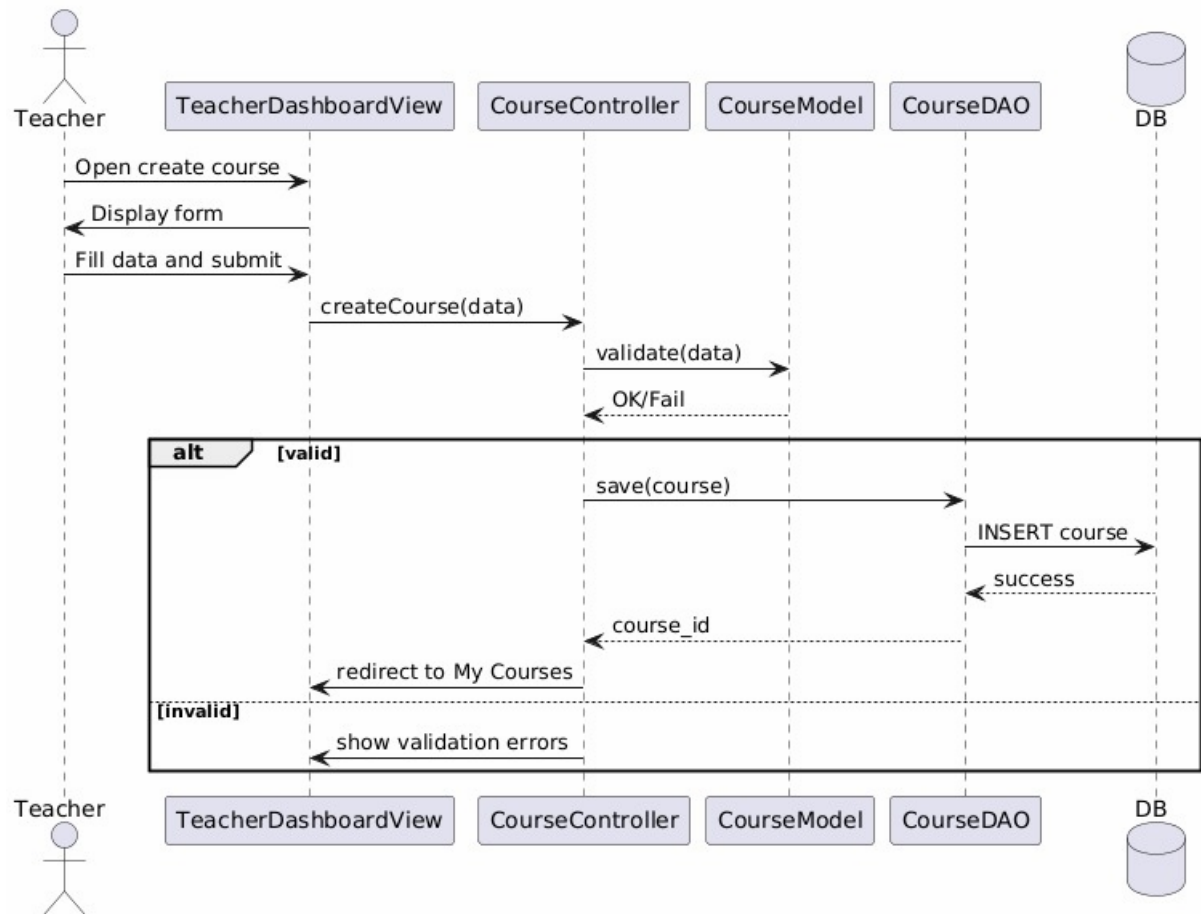
<https://drive.google.com/file/d/13yu7-M-SJFe8fPSzTOVcUkoN18AqkLze/view?usp=sharing>



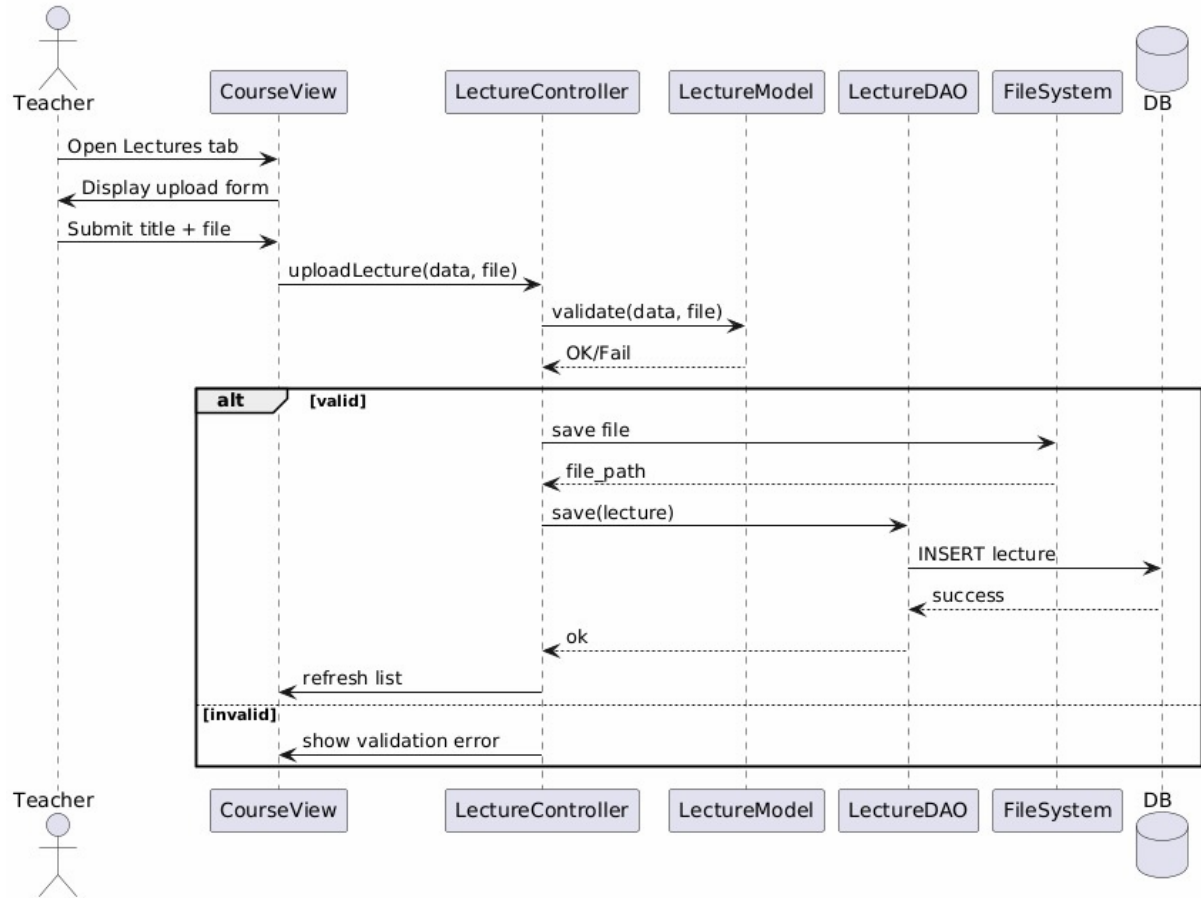
4.2.3 Sequence Diagrams//



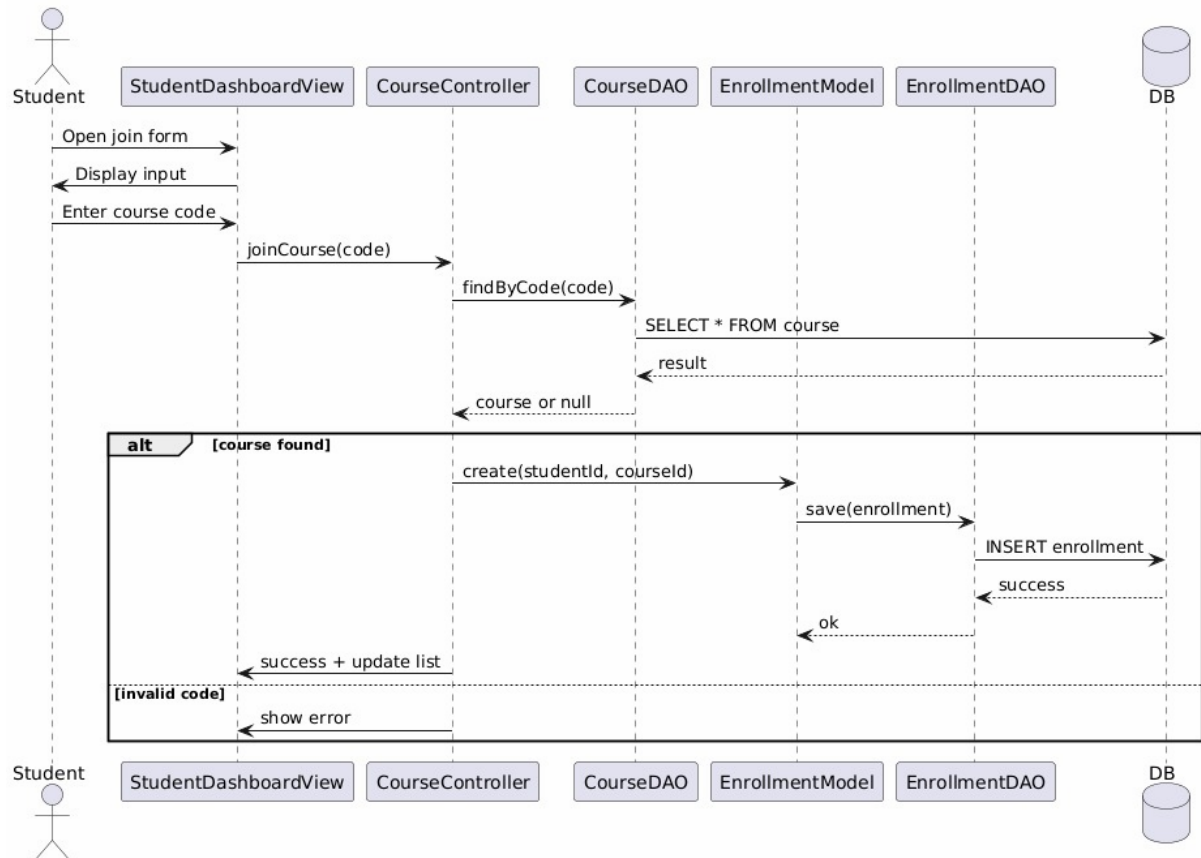
UC2 - Create Course



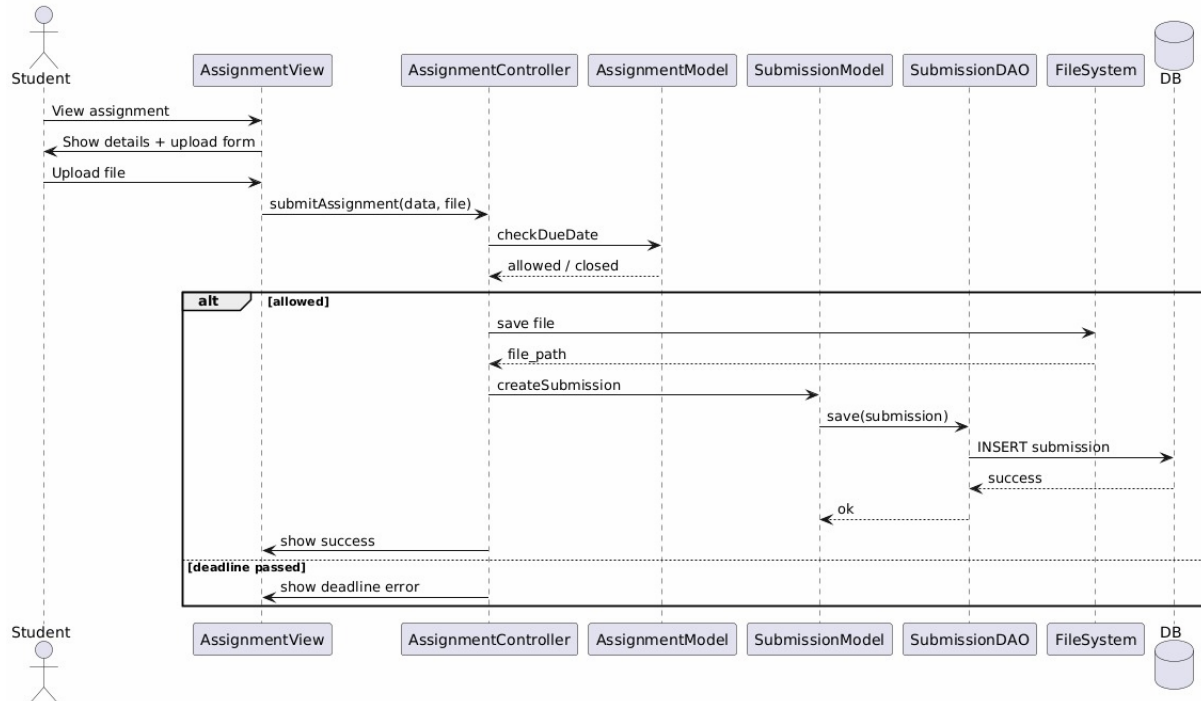
UC4 - Upload Lecture



UC3 - Join Course by Code



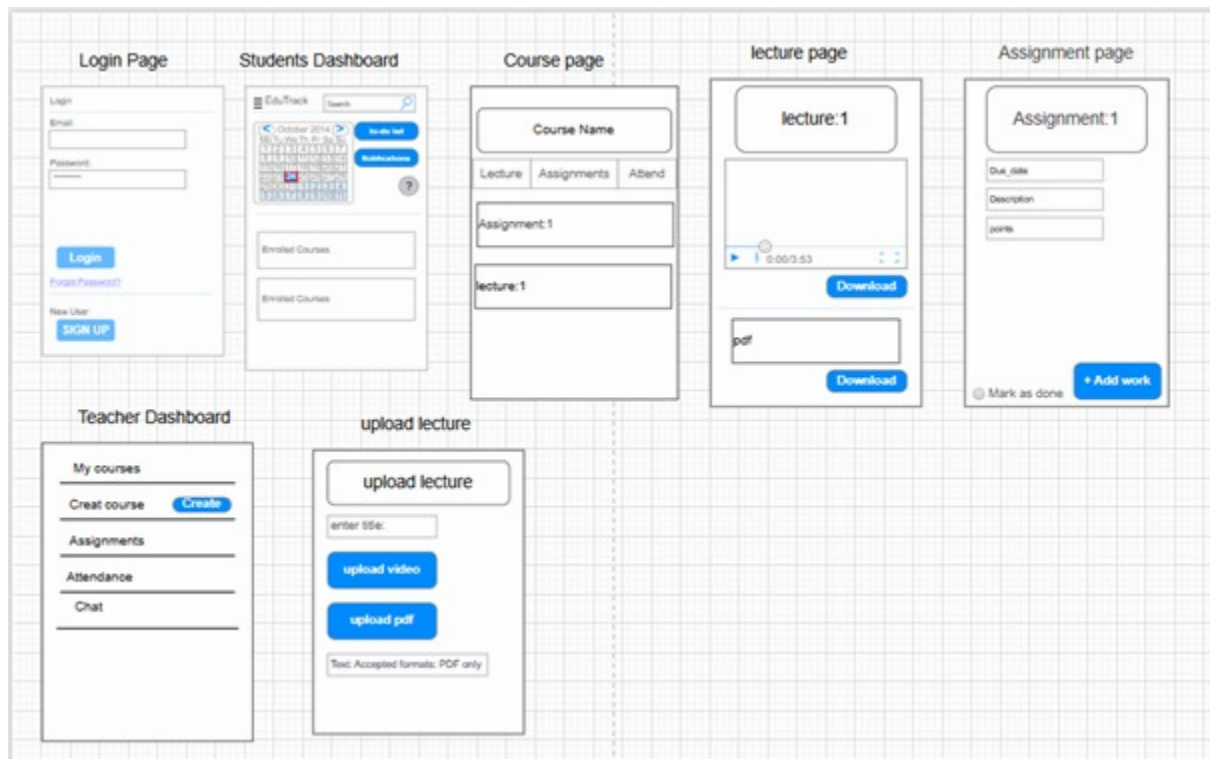
UC5 - Submit Assignment



4.3 UI/UX Design

4.3.1 Wireframes

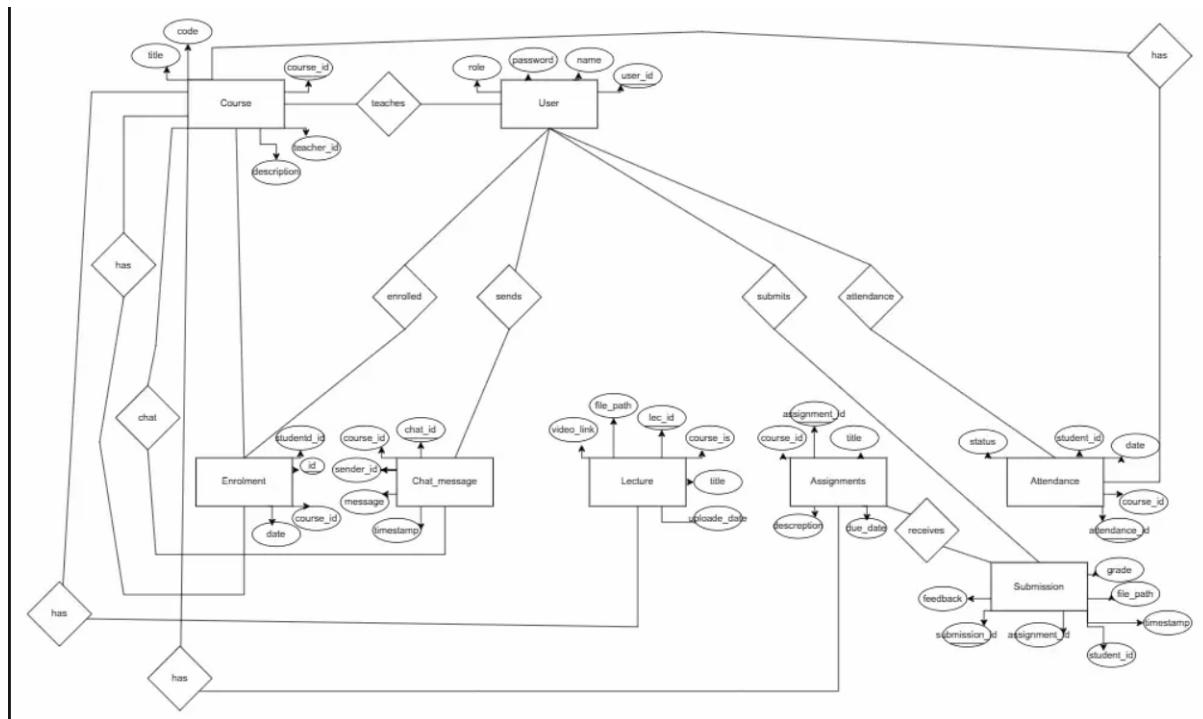
https://drive.google.com/file/d/1UjB-WCmmxT_XFo4UVIV11WBtE-eg7tKv/view?usp=sharing



4.4 Data Design

4.4.1 Database Schema / ER Diagram

https://drive.google.com/file/d/1iLouvWXSDeqqS63dWAMuRkZ_X5NNQ-/view?usp=sharing



4.4.3 Data Dictionary

User Table

Note: In the class diagram, Student, Teacher, and Admin inherit from User. However, in the database all roles are stored in this table.

Column	Data Type	Description / Purpose
user_id	INT (PK)	Unique identifier for each user.
name	VARCHAR	Full name of the user.
email	VARCHAR	User login email; must be unique.
password	VARCHAR	Hashed password for secure authentication.
role	ENUM	indicates user type: student, teacher, admin (used for inheritance logic).

Course Table

Column	Data Type	Description / Purpose
course_id	INT (PK)	Unique identifier for each course.
teacher_id	INT (FK)	References User.user_id of the teacher who owns the course.

title	VARCHAR	Course title displayed to users.
code	VARCHAR	Join code used by students to enroll in the course.
description	TEXT	Course description and overview.

Enrollment Table

Column	Data Type	Description / Purpose
id	INT (PK)	Unique identifier for each enrollment record.
student_id	INT (FK)	References User.user_id for the enrolled student.
course_id	INT (FK)	References Course.course_id for the enrolled course.
enrolled_date	DATE	Date when the student joined the course.

Lecture Table

Column	Data Type	Description / Purpose
lecture_id	INT (PK)	Unique identifier for each lecture.
course_id	INT (FK)	References Course.course_id to link lecture to a course.
title	VARCHAR	Title or topic of the lecture.
file_path	VARCHAR	Path on the server to the uploaded lecture PDF file.
video_link	VARCHAR	Optional URL to an external video resource.
upload_date	DATE	Date the lecture was uploaded.

Assignment Table

Column	Data Type	Description / Purpose
--------	-----------	-----------------------

assignment_id	INT (PK)	Unique identifier for each assignment.
course_id	INT (FK)	References Course.course_id for the course that owns the assignment.
title	VARCHAR	Assignment title.
description	TEXT	Detailed description of the task.
due_date	DATE	Deadline for submission.
max_grade	INT	Maximum possible grade for the assignment.

Submission Table

Column	Data Type	Description / Purpose
submission_id	INT (PK)	Unique identifier for each submission.
assignment_id	INT (FK)	References Assignment.assignment_id.
student_id	INT (FK)	References User.user_id for the student who submitted.
file_path	VARCHAR	Path to the uploaded submission file on the server.
timestamp	DATETIME	Date and time when the submission was uploaded.
grade	INT	Grade given by the teacher (nullable until graded).
feedback	TEXT	Textual feedback from the teacher (nullable).

Attendance Table

Column	Data Type	Description / Purpose
attendance_id	INT (PK)	Unique attendance record identifier.
course_id	INT (FK)	References Course.course_id.
student_id	INT (FK)	References User.user_id for the student.
date	DATE	Session date.
status	ENUM	Attendance status: Present, Absent, or Excused.

ChatMessage Table

Column	Data Type	Description / Purpose
chat_id	INT (PK)	Unique identifier for each chat message.
course_id	INT (FK)	References Course.course_id indicating the course chat.
sender_id	INT (FK)	References User.user_id of the message sender.
message	TEXT	Content of the chat message.
timestamp	DATETIME	Date and time when the message was sent.

5. Conclusion

5.1 Summary of Design Phase

The design stage of the EduTrack system specified how the software will operate internally based on requirements collected in the SRS. This document describes a complete architecture following the MVC pattern and includes definitions for system components, models, controllers, and view. This document also describes the behaviour and interactions of the EduTrack system and includes sequence diagrams, detailed class diagram, wireframe mock-ups. The data layer has been defined through a complete database schema and data dictionary. The results of this phase can be used in the implementation phase and will ensure that the system can be modular, maintainable, scalable, and satisfy all the requirements.